CEC 322 Lab 9: Condition Code Suffices and Conditional Branch

Jianhua Liu

Spring 2026

Contents

9.1 Introduction to the project 1
9.2 Provided base code 1
9.3 Project assignments 2
9.3.1 CET: Getting the APSR flags from the `uint64_t val_apsr` variable (10 pts) . . 2
9.3.2 PT1: Obtaining the CCSs based on the APSR flags (30 pts) 3
9.3.3 PT2: Finding the larger of two variables in ASM (15 pts) 4
9.3.4 PT3: Finding the sum of squares of the numbers in a range in ASM (35 pts) 5
9.4 Submission of Lab Report (10 pts) 6

9.1 Introduction to the project

We have learned in the class how the CMP instruction will impact the status flags. We have also learned how the status flags are combined using different logic expressions to form different condition code suffices (CCS) which are used to form conditional branch instructions.

You have been given a number of HW problems to practice the above topics. In this project, we will program similar problems so that they can be tested using Unity. Please read questions of HW 8 carefully before proceeding.

With the conditional branch instructions, we can easily write assembly functions which can implement the `if-else` and `for-while` constructs of C.

9.2 Provided base code

A buildable project for this project is provided, together with some base code. Please see `fo4b_ccs_n_cond_branch.zip`.

The provided base code are divided into two parts:

- Part 1 contains functions we have used in demo project `fm1s_status_flags_in_c.zip`. These functions are needed since we will use status flags to determine the condition code suffices.
- Part 2 includes new functions of this project. They are listed in the sequel one by one, together with related project assignments.

The source files we are working on are:

- `fo4b_ccs_n_cond_branch_cfns.c`.
- `fo4b_ccs_n_cond_branch_sfns.s`

9.3 Project assignments

The project assignments consist of code explanation tasks (CET) and programming tasks (PT), as listed below.

9.3.1 CET: Getting the APSR flags from the `uint64_t val_apsr` variable (10 pts)

The following function, coming from `fm1s_status_flags_in_c`,

```
uint64_t mp_n_bit_x_minus_y_with_apsr(int32_t x, int32_t y, int n);
```

can be used to return the `uint64_t val_apsr` variable according to the values of `x`, `y`, and `n`. In this 64-bit return variable; the lower word contains the value of `n`-bit subtraction of `x` and `y`, and the upper word contains the value of the **simulated APSR** register.

This *APSR register* is defined using bit-field, a user data type `APSR_t` defined in `fo5a_ccs_n_cond_branch.fns.h`. To get the value of the APSR type out of `val_apsr`, we can use the following function:

```
APSR_t mp_val_apsr_to_apsr(uint64_t val_apsr) {
    APSR_t apsr = *((APSR_t *)&val_apsr + 1);
    return apsr;
}
```

Note that in the above code, we have to use a special C trick to get the value out from the upper word of the 64-bit value to a bit-field variable since the assignment of a 32-bit value to a bit-field variable is not directly doable.

This function is tested to work with the following test function

```
void test_mp_val_apsr_to_apsr(void) {
    uint64_t val_apsr0 = 0;
    uint64_t val_apsr1 = ((uint64_t)15) << (32+28);
    uint64_t val_apsr2 = ((uint64_t)10) << (32+28);
    APSR_t apsr0 = mp_val_apsr_to_apsr(val_apsr0);
    APSR_t apsr1 = mp_val_apsr_to_apsr(val_apsr1);
    APSR_t apsr2 = mp_val_apsr_to_apsr(val_apsr2);

    uint32_t exp[] = {0, 0xF0000000, 0xA0000000};
    uint32_t act[3];

    act[0] = *(uint32_t *)&apsr0;
    act[1] = *(uint32_t *)&apsr1;
    act[2] = *(uint32_t *)&apsr2;

    TEST_ASSERT_EQUAL_UINT32_ARRAY(exp, act, 3);
}
```

Of course, this test should pass.

Now, we have the CET: **Explain** the following in your report (10 pts):

- How is the value of APSR assigned to variable `apsr` in `mp_val_apsr_to_apsr`? Specifically, how the type-casting and offset work?
- How or why `uint64_t val_apsr1= ((uint64_t)15)<< (32+28);` is related to the value of `exp[1]`?

Try to explain the code by yourself; ask the instructor in the class if you have trouble.

9.3.2 PT1: Obtaining the CCSs based on the APSR flags (30 pts)

In Lctr 19, we have discussed how to form the CCSs from the APSR flags. Here, we use a data structure defined in `fo4b_ccs_n_cond_branch.fns.h` to hold CCSs, as shown below:

```
typedef struct {
    uint8_t EQ;
    uint8_t NE;

    uint8_t LT;
    uint8_t LE;
    uint8_t GE;
    uint8_t GT;

    uint8_t LO;
    uint8_t LS;
    uint8_t HS;
    uint8_t HI;
} CCS_t;
```

We use a `uint8_t` value for each CCS so that we can do a unit test easily down the road by converting this data structure into an array of `uint8_t` numbers.

The logic expressions used to express CCSs based on status flags are provided in Lctr 19. You need to complete the code of the following function

```
void mp_apsr_to_ccs(APSR_t apsr, CCS_t *pCcs) {

}
```

so that you will have the corresponding CCSs calculated correctly.

The test of the `mp_apsr_to_ccs` function is done with the following two test functions:

Test function 1:

```
void test_mp_apsr_to_ccs_case1(void) {
    int N = 5;
    mp_initialize_n_bit_const(N);

    int x = -3;
    int y = 13;
    uint64_t val_apsr = mp_n_bit_x_minus_y_with_apsr(x, y, N);
    APSR_t apsr = mp_val_apsr_to_apsr(val_apsr);

    uint8_t exp[] = {0, 1,   1, 1, 0, 0,   0, 0, 1, 1}; // Example 1, Lctr 19
    CCS_t ccs;
    mp_apsr_to_ccs(apsr, &ccs);
```

```

    uint8_t *act = (uint8_t *)&ccs;

    TEST_ASSERT_EQUAL_UINT8_ARRAY(exp, act, 10);
}
```

For this test function, we consider `N` = 5. We use the results of Example 1 in Lctr 19.

- We see that they are not the same, so, we have the following CCS values: EQ = 0, NE = 1.
- If we see `x` and `y` as signed numbers, they will be -3 and 13, respectively, so `x` is less than `y`. As such, will have the following CCS values: LT = 1, LE = 1, GE = 0, GT = 0.
- If we see `x` and `y` as unsigned numbers, they will be -3+32=29 and 13, respectively, so `x` is higerer than `y`. As such, will have the following CCS values: LO = 0, LS = 0, HS = 1, HI = 1.

Putting the above together, we have the expected values in `exp[]`.

Note that we can *see* `ccs` as an array of 10 `uint8_t` elements.

Test function 2:

```
void test_mp_apsr_to_ccs_case2(void) {
    int n = N;
    mp_initialize_n_bit_const(N);

    int x = 17; // -15
    int y = 17; // -15
    uint64_t val_apsr = mp_n_bit_x_minus_y_with_apsr(x, y, N);
    APSR_t apsr = mp_val_apsr_to_apsr(val_apsr);

    uint8_t exp[] = {1, 0,   0, 1, 1, 0,   0, 1, 1, 0};
    CCS_t ccs;
    mp_apsr_to_ccs(apsr, &ccs);

    uint8_t *act = (uint8_t *)&ccs;

    TEST_ASSERT_EQUAL_UINT8_ARRAY(exp, act, 10);
}
```

For this test function, we again have `N` = 5. This is just a new hand calculation example.

- We see that they are the same, so, we have the following CCS values: EQ = 1, NE = 0.
- If we see `x` and `y` as signed numbers, they will all be -15. As such, will have the following CCS values: LT = 0, LE = 1, GE = 1, GT = 0.
- The same is true for unsigned number, so we have: LO = 0, LS = 1, HS = 1, HI = 0.

Putting the above together, we have the expected values in `exp[]`.

9.3.3 PT2: Finding the larger of two variables in ASM (15 pts)

In Lctr 19, we have discussed how to implement a ternary operator in both C and assembly. Here, we change the problem to finding the larger (max) of the two input variables, as described below.

```
@ int mp_max_ab_i_c(int a, int b) {
@     if (a >= b) {
```

```
@         return a;
@     } else {
@         return b;
@     }
@ }
@ int mp_max_ab_i_s(int a, int b)
@   Input:
@     * a -> r0
@     * b -> r1
@   Output:
@     * r0
mp_max_ab_i_s:

mp_max_ab_i_s_end_if:
    bx      lr
```

You need to finish the programming of this function to pass the following tests.

```
void test_mp_max_ab_i_s(void) {
    int a[] = {0xFFFFFFFF, 0xFF, 0};
    int b[] = {0,             0, 0};
    int exp[] = {0,        0xFF, 0};
    int act[3];

    for (int i = 0; i < 3; i++) {
        act[i] = mp_max_ab_i_s(a[i], b[i]);
    }
    TEST_ASSERT_EQUAL_INT32_ARRAY(exp, act, 3);
}
```

9.3.4 PT3: Finding the sum of squares of the numbers in a range in ASM (35 pts)

In Lctr 19, we have also discussed how to program a simple loop in assembly to find the sum of numbers in a range. Here, we change the problem to finding the sum of squares of the numbers in a range. You need to finish the function below to pass the test function.

The function is given below.

```
@ int64_t mp_range_square_sum_standard_while_c(int s, int e) {
@     int64_t sum = 0;
@     int64_t i = s;
@     while (i <= e) {
@         sum += i*i;
@         i++;
@     }
@     return sum;
@ }
@ int64_t mp_range_square_sum_standard_while_s(int s, int e)
@   Input:
@     * s -> r0 (will be used as i)
@     * e -> r1
@   Output:
@     * r1:r0
@   Others:
@     * sum_low_word -> r2, sum_high_word -> r3
mp_range_square_sum_standard_while_s:
```

```

    bx      lr                      @ return sum
```

The test function is given below:

```
void test_mp_range_square_sum_standard_while_s(void) {
    int s[] =       {3, -3,  0, 0x100000};
    int e[] =       {0,  0,  3, 0x100000};
    int64_t exp[] = {0, 14, 14, 0x10000000000};
    int64_t act[4];

    for (int i = 0; i < 4; i++) {
        act[i] = mp_range_square_sum_standard_while_s(s[i], e[i]);
    }
    TEST_ASSERT_EQUAL_INT64_ARRAY(exp, act, 4);
}
```

Note that we can find the above sum of squares in a range based on the following formula:

$$\sum_{i=1}^{n} i^2 = n(n+1)(2n+1)/6,$$

the derivation of which can be found at https://www.youtube.com/watch?v=aI0M4XRiz4I. We just assume we don't know the above formula and have to resort to using programming to calculate the result.

9.4 Submission of Lab Report (10 pts)

The lab report should at least include the following.

- An explanation for Code explanation task 1.
- The code you wrote for the C and assembly functions to pass the tests in Programming tasks 1 to 3.
- A screenshot showing that you have passed the tests.

The report and zip file names should be something like:

- `fo4b-report-lastname-firstname.pdf`.
- `fo4b-proj-lastname-firstname.zip`.